

COMP1047 Lab Week 02 - Solution

Part 1: Computer Performance

Q1. Consider three different processors P1, P2, and P3, executing the same set of instructions. P1 has 3GHz clock rate and a CPI of 1.5; P2 has a 2.5GHz clock rate and a CPI of 1.0; P3 has a 4.0GHz clock rate and has a CPI of 2.2.

(i) Which processor has the highest performance expressed in instructions-per-second?

Solution

$$IPS_1 = CR_1/CPI_1 = 3GHz/1.5 = 2 * 10^9$$

$$IPS_2 = CR_2/CPI_2 = 2.5GHz/1.0 = 2.5 * 10^9$$

$$IPS_3 = CR_3/CPI_3 = 4GHz/2.2 \approx 1.8 * 10^9$$

Hence the processor P2 has the highest performance in terms of instructions per second.

(ii) If the processor each execute a program in 10 seconds, find the number of cycles and the number of instructions.

Solution

$$numOfCycles_1 = 10sec \times ClockRate_1 = 10 * 3 * 10^9 = 3 * 10^{10}$$

$$numOfInstr_1 = numOfCycles_1/CPI_1 = 2 * 10^{10}$$

$$numOfCycles_2 = 10sec \times ClockRate_2 = 10 * 2.5 * 10^9 = 2.5 * 10^{10}$$

$$numOfInstr_2 = numOfCycles_2/CPI_2 = 2.5 * 10^{10}$$

$$numOfCycles_3 = 10sec \times ClockRate_3 = 10 * 4 * 10^9 = 4 * 10^{10}$$

$$numOfInstr_3 = numOfCycles_3/CPI_3 = 1.8 * 10^{10}$$

(iii) We are trying to reduce the execution time by 30% but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

Solution

$$time = \frac{numOfInstr * CPI}{CR} \rightarrow new_CR_ratio = 120\%/70\% \approx 1.7$$

$$CR_1^{new} = CR_1 * 1.7 = 5.1GHz$$

$$CR_2^{new} = CR_2 * 1.7 = 4.3GHz$$

$$CR_3^{new} = CR_3 * 1.7 = 6.8GHz$$

Q2. Compilers can have profound impact on the performance of an application. Assume that for a program, compiler A results in a dynamic instruction count of 1.0E9 and has an execution time of 1.1s, while compiler B results in a dynamic instruction count of 1.2E9 and an execution time of 1.5s.

(i) Find the average CPI for each program given that the processor has a clock cycle time of 1ns.

Solution

$$CPI_A = 1.1s * 10^9 / (1.0E9) = 1.1$$

$$CPI_B = 1.5s * 10^9 / (1.2E9) = 1.25$$

(ii) Assume the compiled programs run on two different processors. If the execution times on the two processors are the same, how much faster is the clock of the processor running compiler A's code versus the clock of the processor running compiler B's code?

Solution

$$CPUTime = \frac{ClockCycles}{ClockRate} = \frac{InstrCount * CPI}{ClockRate}$$

$$\text{Same CPU time, hence } \frac{InstrCount_A * CPI_A}{ClockRate_A} = \frac{InstrCount_B * CPI_B}{ClockRate_B}$$

$$\text{We have } \frac{ClockRate_A}{ClockRate_B} = \frac{InstrCount_A * CPI_A}{InstrCount_B * CPI_B} = \frac{1.0E9 * 1.1}{1.2E9 * 1.25} = 0.73$$

So running on compiler A is 27% slower than B.

(iii) A new compiler is developed that uses only 6.0E8 instructions and has an average CPI of 1.1. What is the speedup of using this new compiler versus using compiler A or B on the original processor?

Solution

$$CPUTime_{new} = \frac{InstrCount * CPI_{new}}{ClockRate}$$

$$\frac{CPUTime_{new}}{CPUTime_A} = \frac{InstrCount_{new} * CPI_{new}}{InstrCount_A * CPI_A} = \frac{6.0E8 * 1.1}{1.0E9 * 1.1} = 0.6$$

$$\frac{CPUTime_{new}}{CPUTime_B} = \frac{InstrCount_{new} * CPI_{new}}{InstrCount_B * CPI_B} = \frac{6.0E8 * 1.1}{1.2E9 * 1.25} = 0.44$$

So the speedup compared to A or B is 40% or 56%.

Part 2: Getting Familiar with QtSpim

We will program the MIPS assembly code using an IDE named QtSpim. Please download and install it at (<https://sourceforge.net/projects/spimsimulator/files/>). Some useful information can be found at (<http://spimsimulator.sourceforge.net/>). You may also see that QtSpim has already been installed on the computers in our lab room.

Try your first MIPS program

Once you are familiar with the software, you can now try to execute some simple MIPS programs. The first one you could try is the famous "Hello World", which almost every programmer will use as their first program. To do this, firstly, create a new text file and named it "HelloWorld.asm" or "HelloWorld.s". Open the file using any text editor (e.g. notepad) and copy/paste the following text into the file:

```
.data
msg: .asciiz "Hello world!\n"
.text
.globl main
main:
    la $a0, msg    #load label msg
    li $v0, 4       #load immediate
    syscall        # print it
    li $v0, 10
    syscall        # exit
```

Save the file before you close the text editor. Now load the program that you just created into QtSpim by clicking the Load File in File menu. Click the Run/Continue button (i.e. the triangle icon) on the menu bar and observe what you get in the console window.

In the next part, you are supposed to write some MIPS code using the instructions learnt in this week's lecture. But before that, you are strongly suggested to:

- View the demo video from Moodle page ([QtSpim_Demo](#)), to familiarize yourself with the QtSpim tool and some example MIPS code.
- Look at the "[QtSpim Syscall](#)" file in the Moodle page to get more experience from the syscall that is used together with your MIPS instructions.

At the end of this lab manual, find some common Q&A regarding the QtSpim.

Part 3: MIPS Programming Questions

Q1. Write a MIPS program to load two integers, 45 and 1026, into registers \$s0 and \$s1 as unsigned numbers. You can place the binary value directly in the data memory segment and then use `lw` instruction to load them into registers. For example, the following program stores two unsigned integers $0000000A_{16}$ and 10000000_{16} in the data segment of the memory, and then loads the first integer in \$s0 using `lw` instruction.

```
        .data
uint:   .word 0x0000000A 0x10000000
        .text
        .globl main
main:
        la $t0, uint          #load the base address
        lw $s0, 0($t0)        #load the first integer into $s0
```

Note that here we use assembler directive **.word**. It means each number in the following is treated as a 32-bit word. Now print out both numbers to the console using the **syscall** function. Check the output to see whether it is expected.

Solution

In QtSpim, change the register to binary,
run the program step by step, and see the value change
in R16 [s0], R17 [s1] and R4[a0].

```
        .data
uint:   .word 45 1026
        # or .word 0x0000002D 0x00000402
        # or .word 101101 10000000010
nl:     .asciiz "\n"
        .text
        .globl main
main:
        la $t0, uint          #load the base address
        lw $s0, 0($t0)        #load the first integer into $s0
        lw $s1, 4($t0)        #load the second integer into $s1

        move $a0, $s0         # move $s0 to $a0 for printing
        li $v0, 1
        syscall

        la $a0, nl            # print a new line
        li $v0, 4
        syscall

        move $a0, $s1         # move $s1 to $a0 for printing
        li $v0, 1
        syscall

        li $v0, 10            # exit
        syscall
```

Q2. Write a program in MIPS assembly language which reads two integer numbers x and y from the console, calculates, then prints $x - 2y - 40$. *Hint: no multiplication is necessary and proper user prompts are expected.*

To read an integer from the console:

```
li $v0, 5    # read_int
syscall
# after syscall, $v0 contains the number just entered
```

To print an integer to the console:

```
# $a0 contains the number to be printed
li $v0, 1    # print_int
syscall
# after syscall, console with print the value contained in $a0
```

Solution

```
.data
prompt1: .asciiz "Please input x: "
prompt2: .asciiz "Please input y: "
rs_string: .asciiz "The result of (x - 2y - 40) is: "
.text
.globl main
main:
    # prompt for input
    la $a0, prompt1 # prompt x
    li $v0, 4
    syscall

    li $v0, 5        # read input x
    syscall

    or $s0, $zero, $v0 # Save x to s0

    la $a0, prompt2  # prompt y
    li $v0, 4
    syscall

    li $v0, 5        # read input y
    syscall

    or $s1, $zero, $v0 # Save y to s1

    la $a0, rs_string # The result is
    li $v0, 4
    syscall
```

```
# calculation
sll $s1, $s1, 1      # 2y
sub $s0, $s0, $s1    # x - 2y
addi $a0, $s0, -40    # a0 = x - 2y - 40

li $v0, 1            # output result
syscall

# exit
li $v0, 10
syscall
```